

System Design Final Project Report

Author: Sujal Pradeep Warke

Roll No: 150096724012

Cohort: Mark Zuckerberg (2024-2028)

Github Repo Link: https://github.com/sujalwarke28/systemDesign_VideoStreaming.git

EC2 Deployment Link: <http://13.232.245.208/>

Problem Statement

The objective was to design and implement a scalable, highly available video streaming platform demonstrating system design principles. The platform must handle large file uploads, serve content via HTTP Range Requests for seeking, and support social features.

Proposed Solution

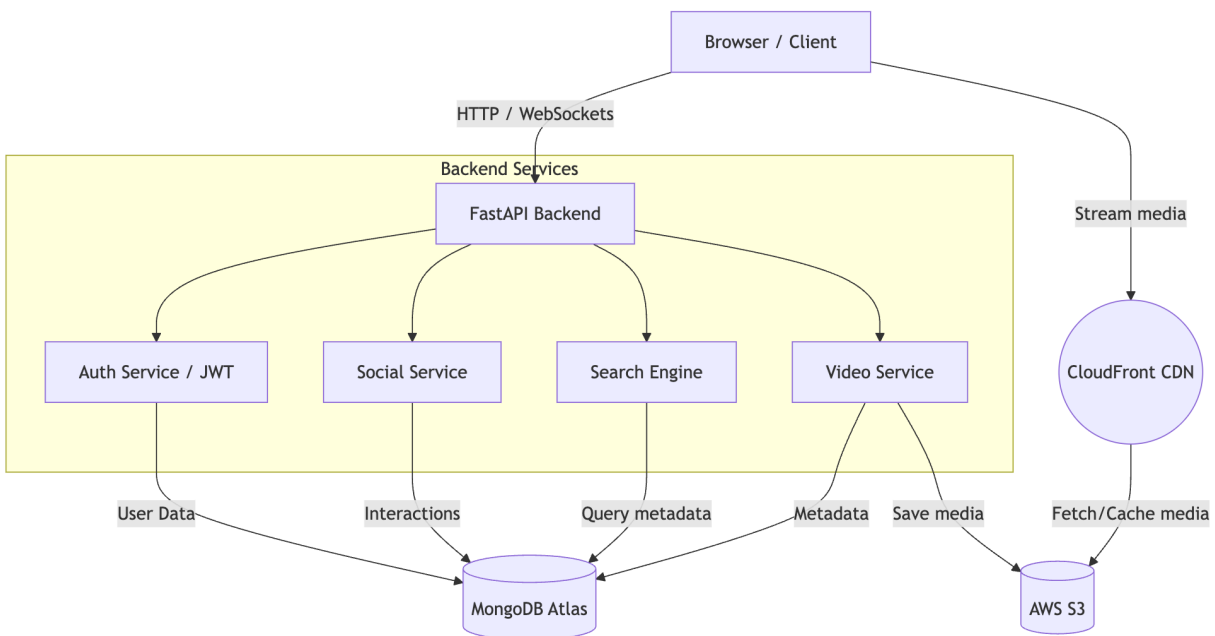
A microservice-ready architecture was adopted, featuring a FastAPI backend hosted on an AWS EC2 instance. MongoDB Atlas manages flexible metadata, while AWS S3 and CloudFront CDN are integrated for highly scalable, global video storage and delivery.

Architecture

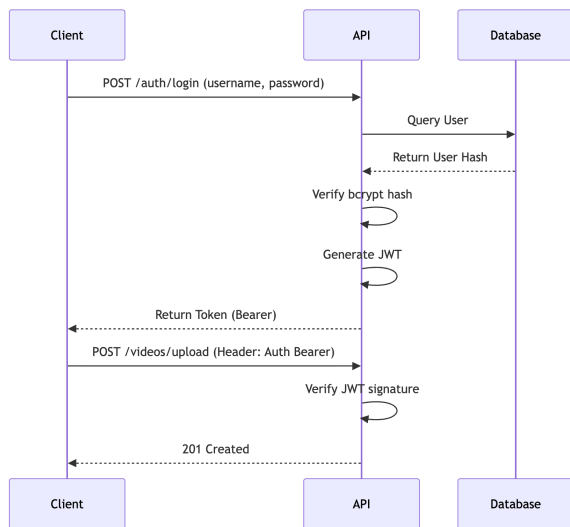
High-Level Architecture (3-Tier Model)

The application follows a standard **3-Tier Architecture**

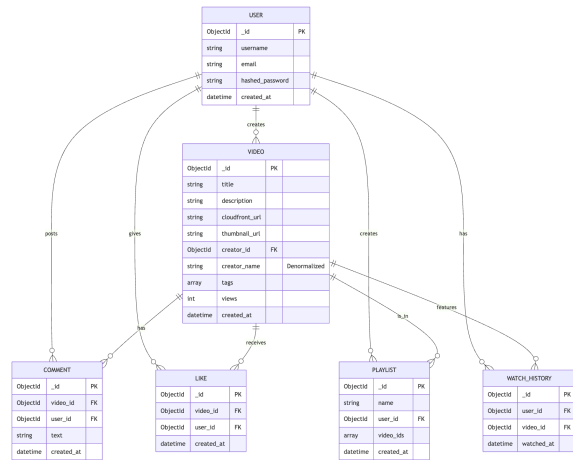
1. **Presentation Layer (Frontend)**: Native HTML5, Bootstrap 5, and Vanilla JavaScript rendered dynamically using FastAPI's Jinja2 templating engine.
2. **Application Layer (Backend)**: Python FastAPI handles all business logic, routing, authentication, and database communications asynchronously. Hosted on AWS EC2 behind an Nginx reverse proxy.
3. **Data Layer (Database & Storage)**: MongoDB Atlas (NoSQL) is used for flexible document storage. AWS S3 acts as the media object storage, and AWS CloudFront serves as the CDN.



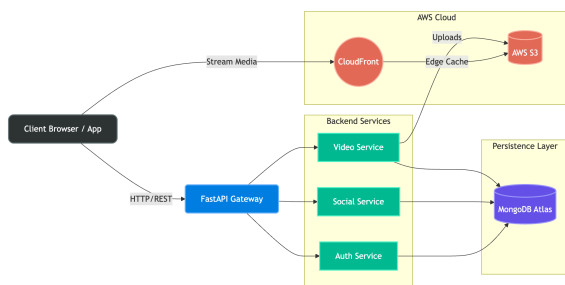
Authentication Flow



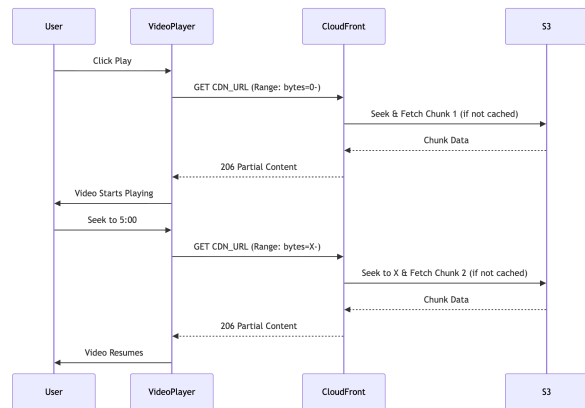
Database ER Diagram



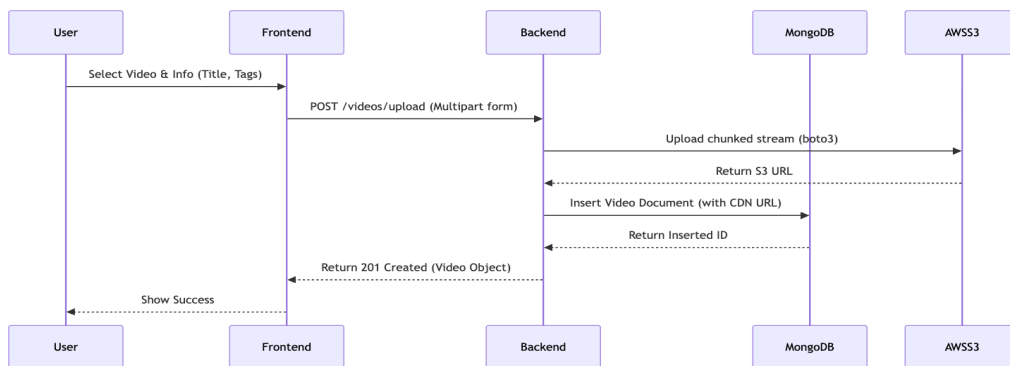
System Architecture



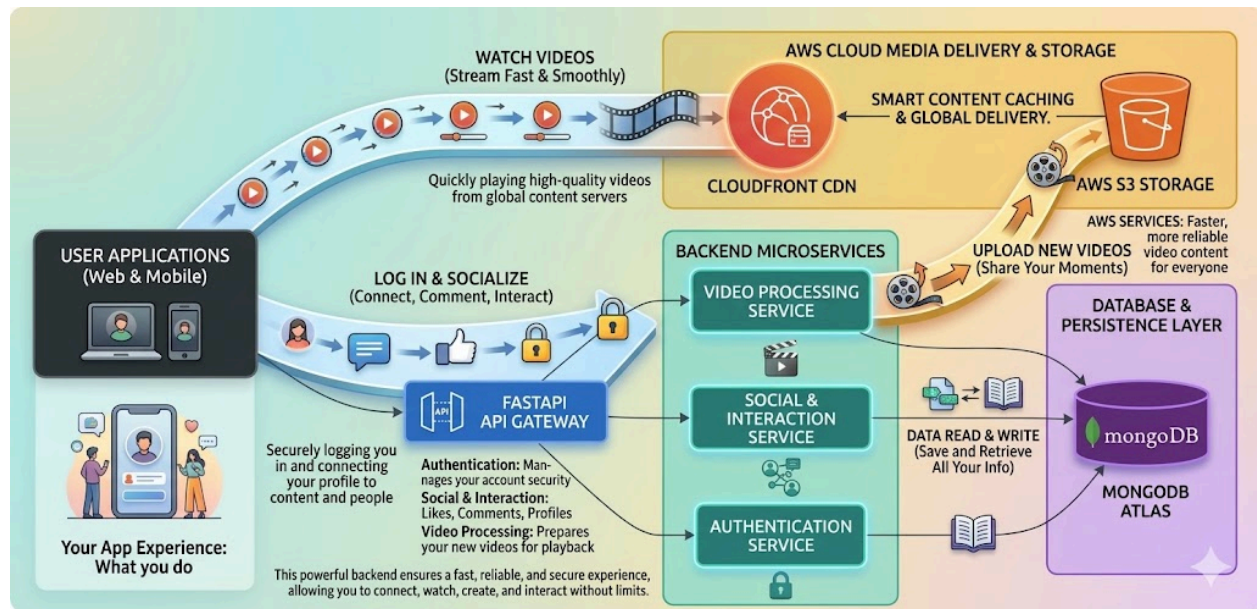
Video Streaming Sequence



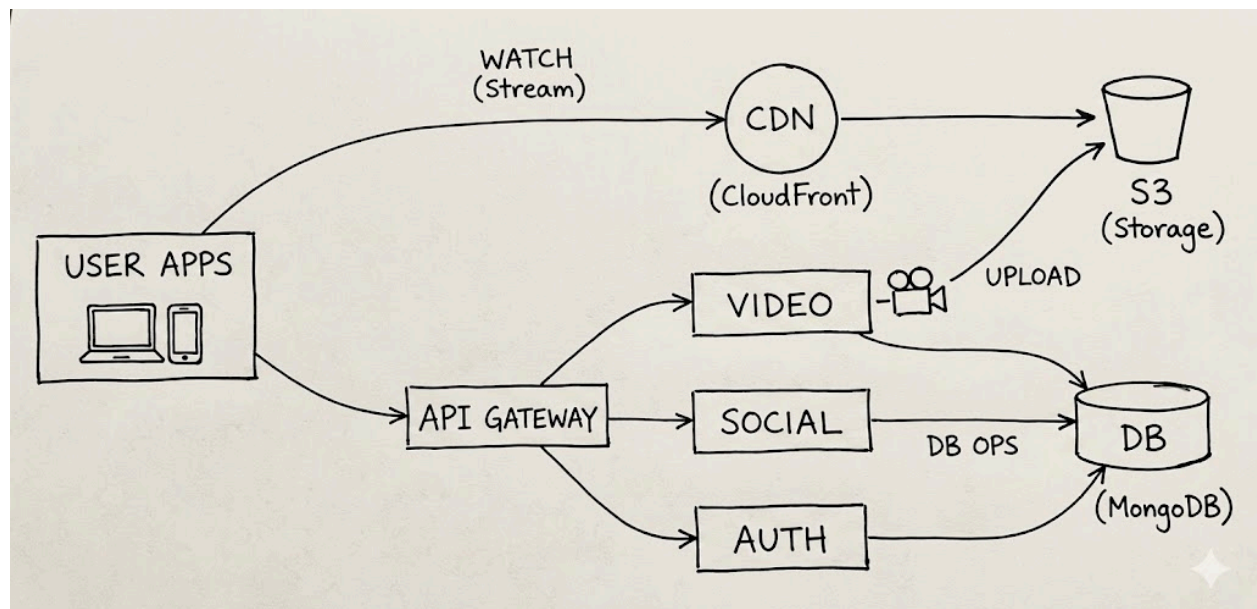
Video Upload Sequence



High Fidelity Diagram



Low Fidelity Diagram



Detailed Technology Stack

1. Frontend Technologies

- **HTML5 & CSS3**: Core markup and styling.
- **Bootstrap 5**: Responsive grid framework and pre-built UI components (Modals, Dropdowns).
- **Vanilla JavaScript**: Handles asynchronous `fetch` requests for liking, commenting, searching, and uploading without requiring page reloads.
- **Jinja2**: Server-side template rendering for injecting dynamic variables (like the page title and API endpoints) directly into the HTML before serving it to the client.

2. Backend Technologies

- **FastAPI**: A modern, fast (high-performance) web framework for building APIs with Python 3.10+ based on standard Python type hints.
- **Uvicorn**: ASGI web server implementation for Python.
- **PyJWT**: For generating and verifying JSON Web Tokens (JWT) used in stateless authentication.
- **Passlib & bcrypt**: For secure password hashing.

3. Database and Storage Technologies

- **MongoDB Atlas**: Cloud-hosted NoSQL database. Documents are stored in BSON format, making it highly flexible for dynamic schemas like Video metadata.
- **Motor**: Asynchronous Python driver for MongoDB, ensuring that database I/O does not block the FastAPI event loop.
- **AWS S3 (Simple Storage Service)**: Acts as our infinitely scalable Object Storage layer for raw media files.
- **AWS CloudFront**: A Content Delivery Network (CDN) that caches S3 videos at global edge locations for ultra-fast, low-latency streaming.

Security Architecture

1. **Stateless Authentication:** Upon login, the backend signs a JWT containing the user's ID and expiration time. The frontend stores this token in `localStorage` and attaches it as a `Bearer` token in the `Authorization` header of all subsequent protected API requests.
2. **Password Hashing:** Plain text passwords are never stored. `bcrypt` salts and hashes the password before insertion into MongoDB.

Data Flow (Video Streaming)

Streaming a video requires special handling to prevent loading a massive file entirely into the server's RAM.

- We utilize **CloudFront Edge Caching** and **HTTP Range Requests** (`206 Partial Content`).
- When the client's video player requests the video, it fetches it directly from the CloudFront CDN URL instead of our EC2 server.
- CloudFront handles the Range headers and seamlessly streams byte chunks from its edge cache, fully offloading the bandwidth and CPU load from our backend API.

System Design Analysis

Q1. Requirements Analysis : Explain the functional and non-functional requirements of the video streaming platform in detail, and justify why each requirement is important for StreamTube.

Answer:

- **Functional Requirements:**
 - **User Authentication:** Secure login using JWT tokens ensures only registered users can upload or interact.
 - **Video Uploads:** Multipart uploads allow users to send large video files and optional thumbnails alongside metadata like titles and tags.
 - **Streaming:** Efficient streaming via HTTP Range Requests allows users to seek and consume partial content without downloading the entire video.
 - **Social Interaction & Playlists:** Likes, comments, and watch history keep users engaged and personalize their experience.
 - **Search & Discovery:** Querying by title/tags and trending algorithms (sorted by view counts) help users find relevant content quickly.
- **Non-Functional Requirements:**
 - **Asynchronous I/O:** Leveraging FastAPI and `motor` (async MongoDB driver) allows the server to handle many concurrent requests without thread blocking.
 - **Minimal Buffering:** Delivering 1MB chunks on demand via HTTP 206 Partial Content avoids high memory overhead and reduces client-side buffering.
 - **Storage Management:** Uniquely identifying files via UUIDs prevents name collisions in the AWS S3 bucket.

Q2. System Architecture Design : Design a high-level architecture for the system and explain how components such as clients, upload service, encoding service, storage system, and content delivery network interact with each other.

Answer:

The system follows a microservice-ready pattern:

- **Presentation Layer (Clients):** A native HTML/JS frontend using Jinja2 templates via FastAPI. It handles rendering and communicates directly with RESTful API endpoints.
- **Application Layer (Services):** A single FastAPI application acts as the API Gateway and routes requests to specific modules (Auth, Video, Search).

- **Storage Layer:** Videos and thumbnails are stored in an AWS S3 object storage bucket for infinite horizontal scaling.
- **Database Layer:** MongoDB Atlas serves as the primary database, managing flexible document schemas for Users, Videos, and Social Interactions.
- **Delivery:** Content delivery is powered by AWS CloudFront, which acts as a CDN to globally cache video chunks at edge locations. Future upgrades will offload video transcoding to Celery workers.

Q3. Video Processing and Delivery : Describe how videos are uploaded, processed into multiple formats, stored, and streamed to users efficiently with minimal buffering.

Answer:

- **Upload & Storage:** Videos are uploaded via a `multipart/form-data` POST request. The file is streamed directly to AWS S3 using the `boto3` SDK to minimize backend memory usage, and saved with a UUID prefix.
- **Processing:** Currently, the platform stores the raw uploaded format without immediate server-side multi-format transcoding.
- **Streaming:** Delivery is handled by the `/videos/stream/{video_id}` endpoint. The server intercepts the HTTP `Range` header, calculates the requested byte range, and seeks the file pointer accordingly. A Python generator reads and yields 1MB chunks, responding with a `206 Partial Content` status and explicitly setting `Accept-Ranges` and `Content-Range` headers. This native HTML5 video player integration ensures smooth buffering and efficient seeking.

Q4. Database Design : Propose a suitable database design using SQL or NoSQL, and explain how video metadata, user data, and viewing history will be stored and managed.

Answer:

The project uses **MongoDB**, a NoSQL database, for its flexible document structure and scalability:

- **Video Metadata:** Stored in a `videos` collection with fields like `_id` (ObjectId), `title`, `filename`, `creator_id`, `tags` (Array of Strings), `views`, and `created_at`.
- **User Data:** Stored in a `users` collection containing `username`, bcrypt-hashed passwords, and profile details.
- **Interactions & History:** Playlists and watch history are stored by associating an array of video ObjectIds with user ObjectIds. Comments are linked to specific video ObjectIds, enabling fast querying of social metrics.

Q5. Algorithm and Implementation : Write and explain a Python-based approach for handling basic video upload processing or streaming logic (simulation).

Answer:

The streaming logic is natively implemented in the FastAPI backend:

```
def range_iterator(path, start, remaining):
    with open(path, "rb") as f:
        f.seek(start)
        chunk_size = 1024 * 1024 # 1MB chunks
        while remaining > 0:
            data = f.read(min(chunk_size, remaining))
            if not data:
                break
            remaining -= len(data)
            yield data

# Setup Response Headers
headers = {
    "Content-Range": f"bytes {byte1}-{byte2}/{file_size}",
    "Accept-Ranges": "bytes",
    "Content-Length": str(length),
}

return StreamingResponse(
    range_iterator(video_path, byte1, length),
    headers=headers,
    status_code=206
)
```

Explanation: This algorithm determines the requested byte range and seeks the file pointer to `start`. It then iteratively reads 1MB chunks using a generator (`yield data`) until the requested size is met. This keeps server RAM usage practically flat regardless of file size.

Q6. Scalability and Fault Tolerance : Explain how the system will scale to support millions of users globally and handle failures such as server downtime or network issues while ensuring uninterrupted streaming.

Answer:

While currently running locally, the architecture supports significant scaling:

- **Stateless Backend:** The use of JWT tokens means the FastAPI application is stateless. It can be horizontally scaled behind a load balancer without sticky sessions.
- **Database Scaling:** MongoDB Atlas supports automatic sharding and replica sets to handle global read/write loads and node failures.
- **CDN Integration:** Video storage is managed by AWS S3 with an AWS CloudFront CDN in front. This caches video chunks at edge locations globally, ensuring uninterrupted streaming during traffic spikes and protecting the EC2 origin server from excessive bandwidth demands.
- **Asynchronous I/O:** The backend uses async endpoints and drivers (`motor`), meaning network delays or slow database queries won't block the main server thread, preventing downtime during high concurrency.

Module Descriptions

- **Auth:** Manages JWT creation and bcrypt hashing.
- **Videos:** Handles multipart uploads, thumbnail generation logic, and partial content delivery.
- **Social:** Allows toggling likes and adding comments.
- **Playlists:** Organizes video curations and tracks watch history.
- **Search:** Executes regex-based queries and tag matching in MongoDB.

Implementation Details

The backend is built in Python 3 using FastAPI. Authentication relies on OAuth2PasswordBearer. Video streaming is implemented via generators yielding byte chunks, explicitly setting `Accept-Ranges` and `Content-Range` headers to support native HTML5 `<video>` player seeking.

Algorithms Used

1. Video Streaming Algorithm (Chunking & Range Requests)

Located in `routes/video_routes.py`, a chunking algorithm is implemented to serve large video files efficiently.

- **How it works:** It intercepts HTTP `Range` headers from the client's video player, calculates the precise `byte1` (start) and `byte2` (end) offsets, and uses a Python generator function (`range_iterator`) to seek the specific spot in the file. It then reads and yields the video data in precise **1MB chunks** (`1024 * 1024 bytes`), responding with a `206 Partial Content` status. This prevents the server from loading massive video files into RAM all at once.

2. Search & Pattern Matching Algorithm

Located in `routes/search_routes.py`, a dynamic search algorithm is implemented using Regular Expressions.

- **How it works:** It takes the user's search query and compiles it into a case-insensitive regex (`re.compile(q, re.IGNORECASE)`). It then dynamically builds a MongoDB query using `$or` logical operators to scan across multiple fields simultaneously (title, description, and creator name). It also intersects this with tag matching using `$and` and `$in` operators if tags are provided.

3. Content Discovery Algorithms (Trending & Randomization)

Located in `routes/video_routes.py`, there are two algorithms to serve content to users:

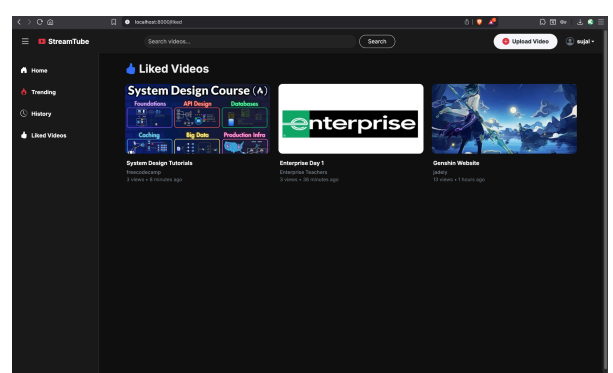
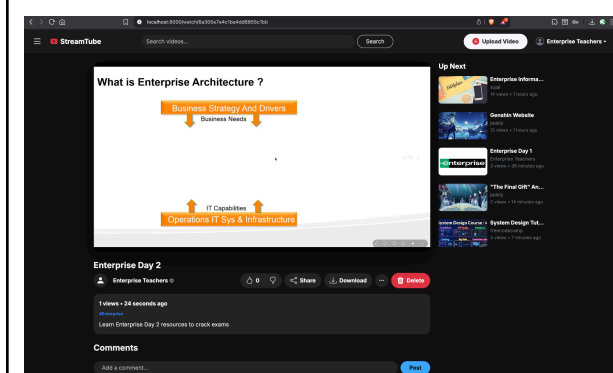
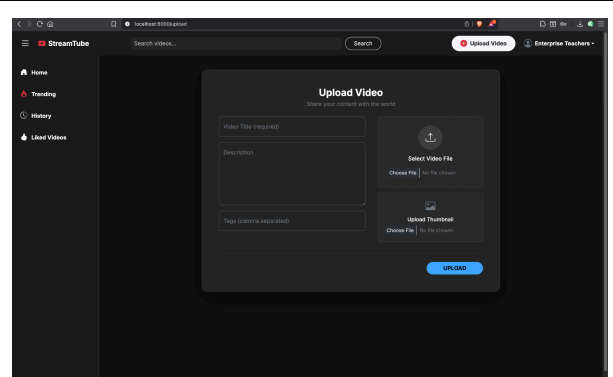
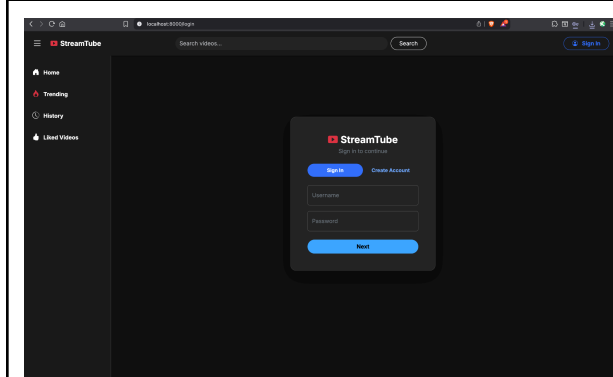
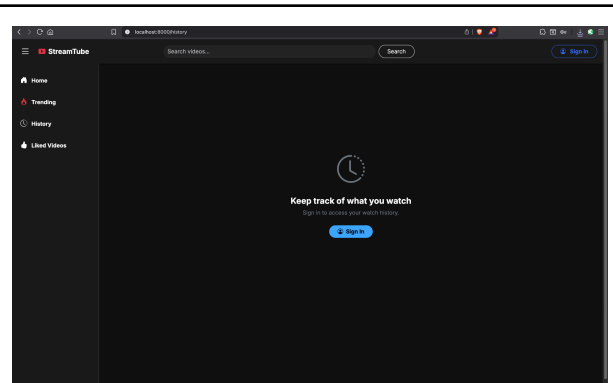
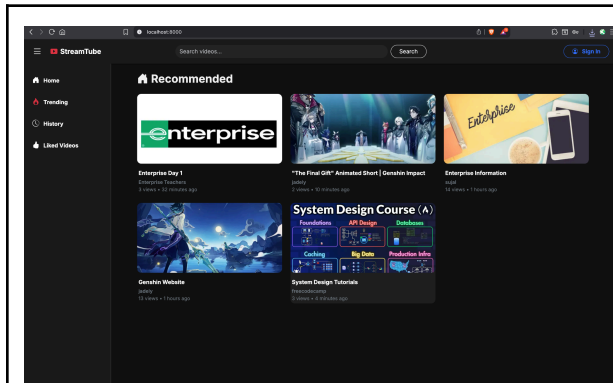
- **Trending (Sorting Algorithm):** It queries the MongoDB database and applies a descending sort based on the integer `views` field (`sort("views", -1)`), limiting the output to the top 20 documents.
- **Randomization (Sampling Algorithm):** It uses MongoDB's built-in `$sample` aggregation pipeline operator (`{"$sample": {"size": 20}}`) to fetch a pseudo-random set of videos to populate the user's feed.

4. Cryptographic Algorithms (Authentication)

Located in `routes/auth_routes.py` and `services/auth_service.py`, standard security algorithms are utilized:

- **Password Hashing:** Uses the **bcrypt** algorithm. It automatically generates a unique cryptographic salt for every user and applies a one-way hash to their password during registration, making it secure against rainbow table attacks.
- **Token Signing:** Uses the **HS256** (HMAC with SHA-256) algorithm to sign the JSON Web Tokens (JWTs). This guarantees the token's integrity and verifies that the stateless sessions are valid and haven't been tampered with by the client.

Screenshots



Future Scope

- **Transcoding**: Offload upload processing to Celery workers for multi-resolution encoding (HLS/DASH).
- **Transcoding**: Offload upload processing to Celery workers for multi-resolution encoding (HLS/DASH).
- **Search Engine**: Migrate text search from MongoDB to Elasticsearch for relevance scoring.